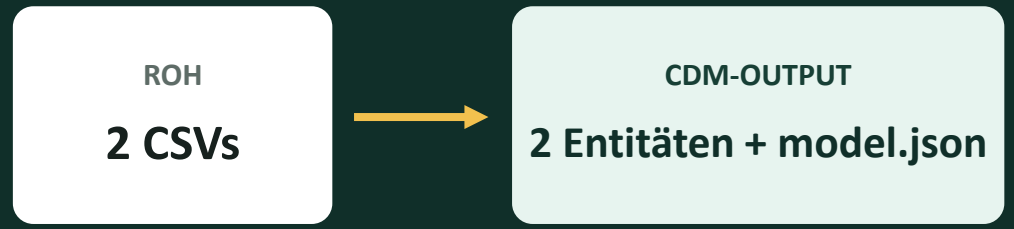
TECHNISCHER WALKTHROUGH

Aus Roh-CSVs werden CDM-fertige Daten



Python parst, transformiert, validiert und lädt das komplette CDM-Paket.

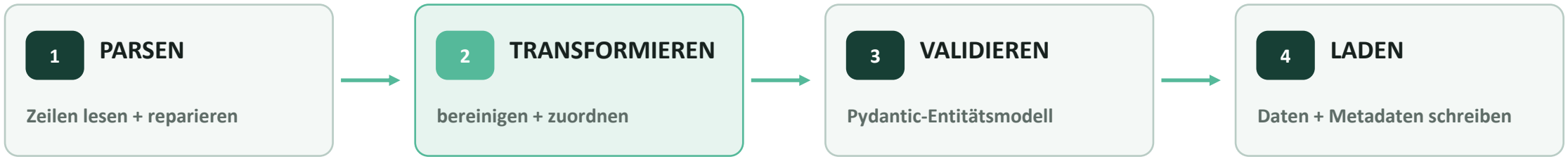
5	Kontakte
4	gültige Aufträge
1	in Quarantäne
ISO	UTC-Zeitstempel

- ✓ Rohzeilen PARSEN
- ✓ Felder TRANSFORMIEREN
- ✓ Modelle + FK VALIDIEREN
- ✓ Daten + Metadaten LADEN

Einstiegspunkt: python -m src.main

Jede Quellzeile parsen und transformieren

Bereinigungsregeln machen aus unregelmäßiger Eingabe typisierte CDM-Felder.





AUTO ein Befehl

MODELL eine Schemaquelle

MENSCHEN Logs + Gründe

python -m src.main

Die Beziehung gültig halten

Jede übernommene SalesOrder verweist auf genau einen Contact.

ELTERN

Contact

contactid PK

firstname · lastname

emailaddress1

createdon · statecode

1 — 0..N

contactid <- customerid

ORD-1005 → CST-999

Kein Contact-Treffer → Quarantäne + Log

KIND

SalesOrder

salesorderid PK

customerid FK

totalamount · decimal

createdon · UTC

Sauberes Ziel Kein defekter Fremdschlüssel

Nachvollziehbar Rohzeile + Grund

LOAD + BESCHREIBEN

Ein Lauf schreibt Daten, Ablehnungen und Metadaten

Die Entitätsmodelle halten CSV-Spalten und model.json-Attribute konsistent.

python -m src.main

- **Contact.csv** — 5 validierte Kontakte
- **SalesOrder.csv** — 4 gültige Aufträge
- **quarantine/*.csv** — Rohzeile + Ablehnungsgrund
- **pipeline-run.log** — Warnungen + Endzahlen

gleiche Modelle →

model.json

Metadaten	Name · Beschreibung · Version
Entitäten	Contact · SalesOrder
Attribute	Name · CDM-Datentyp
Partitionen	relative CSV-Pfade
Beziehung	customerid → contactid

Ergebnis: Ein erneuter Lauf erzeugt dasselbe vollständige CDM-Paket.